

Verbatim Current State Against Roadmap

May to September technical roadmap comparison and next-step plan

Generated from docs/current_state_against_roadmap_may_september.md

Date: 2026-05-23

A readable working report covering the current voice-agent stack, roadmap alignment, future plug-and-play integrations, and evaluation strategy.

Table of Contents

Verbatim Current State Against May to September Roadmap

- Executive Summary
- Current Product State
- Roadmap Alignment
- Most Important Strategic Shift
- Future Integration Strategy
- Integration Platforms to Feature or Support
- Future Tool-Calling Capability Map
- Future Evaluation Strategy
- Recommended Next Steps
- Risk Register
- Updated Roadmap Recommendation
- Source Notes

Verbatim Current State Against May to September Roadmap

Date: 2026-05-23

Roadmap source: /Users/disc/Downloads/_VERBATUM__TECHNICAL_ROADMAP_(MAY__SEPTEMBER).pdf

Executive Summary

Verbatim has moved beyond the exact state described in the May to September roadmap. The roadmap described a first phase centered on core hardening, with tool calling explicitly deferred until after stability. The current local v2 system has already reached a working WebRTC voice-agent baseline with stable custom cascade calls, basic telemetry, a UI dashboard, Google Calendar booking tools through Nango, Twilio SMS follow-up, pre-call knowledge-base injection, and generated post-call summaries.

The most important product conclusion is that Verbatim is no longer only a voice pipeline experiment. It is now an early product-shaped demo platform:

```
LiveKit / Daily / Hume comparison UI
-> Pipecat cascade or Hume direct mode
-> STT, LLM, TTS selection
-> tool calling and integrations
-> call telemetry, transcript, generated summary
```

The strongest custom voice-agent baseline remains:

```
LiveKit -> Deepgram Flux -> Groq LLaMA 3.1 8B Instant -> Cartesia Sonic-3
```

The active .env default currently boots:

```
LiveKit -> Deepgram Nova-3 -> Groq LLaMA 3.1 8B Instant -> Cartesia Sonic-3
```

That distinction matters. The protected benchmark winner is LiveKit + Flux + Groq + Sonic-3, while the environment default may still be Nova-3 unless Flux is selected in the dashboard.

Current Product State

Runtime Modes

The v2 app supports three first-class modes:

1. Pipecat + LiveKit custom cascade
2. Pipecat + Daily custom cascade
3. Hume EVI direct comparison

The current custom cascade is built around:

```
Transport input
-> STT
-> user context aggregation
-> direct tool/action gate
-> LLM
-> identity bleed guard
-> TTS
-> transport output
-> assistant context aggregation
```

The main app is a FastAPI service with one active agent worker at a time. Starting a new call stops the previous worker. This worker isolation was one of the most important stability improvements because it prevents Daily and LiveKit SDK state from contaminating each other in a long-running Python process.

Current Stack

Layer	Current state	Notes
Browser UI	Local dashboard at <code>http://127.0.0.1:8000</code>	Transport/STT/LLM selection, room controls, terminal, transcript, latency, KB, generated call summary
Transport	LiveKit winner, Daily comparison, Hume direct comparison	LiveKit is the best custom-cascade baseline so far
STT	Deepgram Nova-3 and Deepgram Flux	Flux is the protected winning-stack selection; Nova remains a useful stable default/comparison
LLM	Groq LLaMA 3.1 8B Instant default/winner; Gemini/OpenAI/Qwen/xAI/Mock selectable	Groq is fastest usable baseline; no automatic fallback router yet
TTS	Cartesia Sonic-3	Stable and high quality; not the primary current bottleneck
Native voice model	Hume EVI comparison	Useful emotional comparison path, not the protected custom baseline
Tool calling	Google Calendar through Nango; Twilio SMS direct	Booking, availability checks, SMS confirmation
Context	Pre-call dashboard KB	Works as a lightweight knowledge injection path
Call notes	Generated after/from call events	Brief dashboard summary, not LLM prompt input
Telemetry	JSONL events, transcript logs, latency summary, dashboard metrics	Enough for demo debugging; not yet a professional evaluation harness
Tests	74 passing after latest prompt/tool changes	Unit coverage exists around config, events, integrations, rooms, server, static UI

Current User-Facing Behavior

The current real-estate assistant behavior has been narrowed to:

```
Answer the caller's actual question first.
Avoid repeating/rephrasing the caller.
Avoid form-like qualification.
Keep replies short.
After a specific property is discussed, gently offer a property viewing.
After a viewing is booked, offer SMS confirmation.
Do not claim a company, city, or name unless provided.
```

This is a major improvement over the earlier pattern where the agent repeatedly asked rigid qualification questions or over-identified itself.

Current Tool Surface

The LLM is not exposed to raw integration APIs. It sees a small Verbatim-owned tool surface:

```
check_calendar_availability
check_calendar_conflict
prepare_calendar_booking
confirm_calendar_booking
cancel_calendar_booking
send_sms_followup
```

Current policy:

```
Read tools can run without confirmation.
Calendar write tools require explicit confirmation.
```

Only Verbatim-created calendar bookings can be cancelled.
SMS is sent only to the caller phone attached to the call/session.
Provider secrets remain server-side.
Tool events log safe metadata only.

This is the right direction. It is safer than exposing a raw Google Calendar, CRM, or automation catalog directly to the LLM.

Roadmap Alignment

Roadmap Phase 1 - Core Hardening, May

Roadmap goals:

- Reliable STT
- LLM fallback
- Full observability
- Turn handling and interruption reliability

Current status:

Area	Roadmap target	Current state	Status
STT reliability	Keywords, corrections, mis-transcription logging	Nova-3 and Flux are integrated; transcripts are logged; no formal keyword boosting or correction dictionary yet	Partially complete
LLM stability	Gemini primary, Groq fallback	Provider selection exists; Groq is now primary for winning stack; no automatic fallback router yet	Partially complete, changed direction
Observability	STT, LLM input/output, latency, errors	JSONL events, transcript logs, dashboard latency, tool events, terminal logs, generated call summary	Mostly complete for demo
Turn handling	Natural barge-in, no overlaps	Latest v2 reset removed major cutouts/interruption instability; still needs formal benchmark evidence	Mostly complete for demo
Pipeline reliability	No crashes/silent failures	Worker isolation and active-agent kill path improved reliability	Mostly complete for local demo

Assessment:

Phase 1 is demo-stable but not production-complete. The system has enough reliability to support demos and tool experiments, but it still lacks formal STT correction workflows, automatic LLM fallback, and benchmark-grade evidence.

Roadmap Phase 2 - Tool Calling System, June

Roadmap goals:

- Structured tool calling
- Tool registry
- Execution layer
- Booking property viewings
- Fetching listings
- Checking availability
- Sending SMS/WhatsApp follow-ups
- Memory system

Current status:

Area	Roadmap target	Current state	Status
Tool registry	Central execution layer	Verbatim-owned scheduling/follow-up tool surface exists	Started
Calendar booking	Book property viewing	Implemented through Nango Google Calendar	Functional
Availability checks	Check available slots	Implemented	Functional
SMS follow-up	Send confirmation/follow-up	Implemented through Twilio	Functional
Listings	Fetch listings	Not implemented	Future
CRM	Lead/contact/deal updates	Not implemented	Future
Memory	User profile and conversation memory	Only call-scoped state exists	Future
Async execution	Immediate spoken response while tools run	Some direct tool flow exists; timeouts/fallbacks exist; not a mature async tool orchestration model	Partial

Assessment:

Tool calling arrived earlier than the original roadmap allowed. This is good for product validation, but the next step must be making integrations plug-and-play and client-owned. The current tools are a prototype-safe surface, not yet a scalable integration product.

Roadmap Phase 3 - Evaluation Framework, July

Roadmap goals:

- 100-500 simulated conversations
- Task success rate
- Tool execution accuracy
- Latency
- Fallback rate
- Conversation quality
- Replay system

Current status:

Area	Roadmap target	Current state	Status
Internal scenarios	100-500 simulated conversations	Manual tests and local traces only	Not complete
Metrics	Task success, latency, tool accuracy, quality	Latency and tool events exist; no formal task scoring	Partial
Replay	Offline reruns and regression comparison	Not implemented	Future
Professional benchmarks	Research-recognized evaluation	Not implemented	Future

Assessment:

Evaluation is now the largest credibility gap. The dashboard is useful for development, but YC-level credibility requires a separate professional benchmark/evaluation layer.

Roadmap Phase 4 - Telephony Integration, August

Roadmap goals:

- Twilio or Telnix phone calls
- SIP handling
- Call routing
- Multi-call concurrency

Current status:

- Twilio SMS is implemented.
- Twilio voice/SIP is not implemented.
- Telnix is not implemented.
- Multi-call concurrency is not implemented.

Assessment:

Telephony should remain future work until the WebRTC agent and evaluation layer are more stable. SMS support is useful, but it is not the same as phone deployment.

Roadmap Phase 5 - Audio Engineering, Parallel

Roadmap goals:

- Noise suppression
- Echo cancellation
- Audio normalization
- High-quality real-world audio

Current status:

The system has browser/WebRTC metrics and some LiveKit client stats. The major cutout problem improved after the v2 reset and worker isolation. There is no formal audio quality lab yet.

Assessment:

Audio engineering should be handled through benchmark scenarios and client/device telemetry rather than ad hoc knob changes.

Roadmap Phase 6 - Deployment and SaaS, August to September

Roadmap goals:

- Backend
- Session management
- Worker scaling
- Authentication
- API layer
- Dashboard
- Multi-client SaaS

Current status:

- FastAPI backend exists.
- Local dashboard exists.
- Per-call worker process exists.
- One active agent at a time.
- SQLite integration store exists.
- No authentication.
- No hosted deployment.
- No multi-client SaaS control plane.
- No worker queue/autoscaling.

Assessment:

The project is product-shaped but not SaaS-shaped. The most important future SaaS piece is not just auth; it is the client onboarding and integration provisioning flow.

Most Important Strategic Shift

The roadmap originally said:

```
Reliability first
Then capability
Then measurement
Then scale
```

The project has already partially crossed into capability before fully completing reliability and measurement. That is acceptable for a demo, but the next phase should intentionally rebalance:

1. Preserve the working voice baseline.
2. Make integrations plug-and-play.
3. Build real evaluation infrastructure.
4. Then return to telephony and SaaS scaling.

Future Integration Strategy

Goal

The future client experience should be:

```
Verbatim sends the client an onboarding link.
The client fills out a business onboarding form.
The client selects their business type and tools.
The client connects tools through OAuth/connect flows.
Verbatim automatically creates a safe tool configuration.
The voice agent gets only the allowed Verbatim tool surface.
```

This should avoid manual `.env` changes per client. Clients should not need technical help to connect their stack.

Recommended Onboarding Flow

- 1 Verbatim creates a client onboarding link.

```
POST /api/clients/{client_id}/onboarding-link
```

The link should be scoped, expiring, and signed.

- 1 Client chooses business type.

Examples:

```
Real estate agency
Clinic
Restaurant
Home services
Legal office
Recruiting agency
SaaS support
Hotel / hospitality
Automotive dealership
```

- 1 Client selects business goal.

Examples:

```
Book appointments
Qualify leads
Answer FAQs
Route support tickets
Send follow-ups
Create CRM leads
Check order/status data
```

- 1 Client selects stack.

Examples:

- Google Calendar
- Outlook Calendar
- HubSpot
- Salesforce
- Pipedrive
- Google Sheets
- Airtable
- Twilio
- WhatsApp Business
- Gmail
- Slack

1 Client connects tools.

Use OAuth/connect sessions through an integration layer such as Nango first. For tools Nango does not cover, add adapters through Composio, Pipedream, Zapier, direct APIs, or MCP servers.

1 Verbatim maps raw integrations to safe capabilities.

Raw app APIs should never be handed directly to the LLM. Instead, each client gets a small generated capability set:

- check_availability
- book_appointment
- cancel_verbatim_created_appointment
- send_confirmation_sms
- create_or_update_lead
- add_call_note_to_crm
- search_knowledge_base
- handoff_to_human

1 Client reviews permissions.

The onboarding form should show:

- Read-only permissions
- Write permissions
- Confirmation-required actions
- Never-allowed actions
- Data retention settings
- Escalation contact
- Fallback message

1 Verbatim generates a client profile.

Client config should include:

- client_id
- business_type
- business_name
- timezone
- voice/persona
- allowed tools
- connected integrations
- tool policies
- booking rules
- follow-up rules
- knowledge base sources
- handoff rules
- evaluation profile

Recommended Integration Architecture

- Client onboarding form
 - > integration connect sessions
 - > integration connection store
 - > capability mapper
 - > client tool policy
 - > Verbatim tool registry
 - > Pipecat LLM tool schema
 - > tool execution layer
 - > safe response formatter
 - > telemetry/eval logs

Why This Matters

The integration product is not "can the LLM use Google Calendar." That already works.

The real product is:

- Can a non-technical business connect their stack safely in 10 minutes?
- Can Verbatim generate the right tool surface automatically?
- Can Verbatim prevent unsafe writes?
- Can Verbatim measure whether each integration works?

That is the difference between a demo and a deployable SaaS.

Integration Platforms to Feature or Support

Primary Integration Layer

Platform	Why it matters	Recommended role
Nango	OAuth and customer-owned integration connections	Primary integration connection layer
Composio	Tool/action layer for AI agents across many apps	Secondary adapter source for agent tools
Pipedream	Workflows, triggers, actions, and AI/MCP integration surface	Async workflow adapter and long-tail integrations
Zapier AI Actions	Broad app catalog and non-technical automation familiarity	SMB-friendly action layer for common workflows
Make	Visual workflow automation with broad SMB adoption	Future no-code automation bridge
n8n	Open-source/self-hosted workflow automation	Future self-hosted/client-managed option
Workato	Enterprise iPaaS	Later enterprise integrations
Tray.io	Enterprise automation and iPaaS	Later enterprise integrations
Paragon	Embedded integrations for SaaS apps	Future embedded integration alternative
Merge.dev	Unified APIs for HRIS, ATS, accounting, ticketing, CRM categories	Useful for vertical-specific unified data
Apideck	Unified API categories such as CRM/accounting/HRIS	Alternative unified API layer

Business App Integrations to Offer First

Scheduling

Google Calendar
Microsoft Outlook Calendar
Calendly
Cal.com
Acuity Scheduling
OnceHub

CRM and Lead Management

HubSpot
Salesforce
Pipedrive
Zoho CRM
Close
Monday.com CRM
Airtable
Google Sheets
Follow Up Boss
Propertybase

Real Estate Specific

MLS / IDX through RESO Web API where available
Property Finder
Bayut / Dubizzle
Zillow Premier Agent
Realtor.com
Rightmove
Zoopla
LoopNet / CoStar style commercial-property sources
Internal listing database or CSV/Sheet upload

Real estate integrations should be phased carefully. Public listing portals often have limited APIs, partner requirements, or data-use restrictions. The first production-friendly path is likely:

client-owned Google Sheet / Airtable / CRM listing inventory
-> Verbatim KB/listing search
-> viewing booking
-> CRM lead creation
-> SMS/email follow-up

Messaging and Follow-up

Twilio SMS
WhatsApp Business Cloud API
Telnyx Messaging
SendGrid
Postmark
Resend
Gmail
Outlook Mail
Slack
Microsoft Teams

Support and Ops

Zendesk
Intercom
Freshdesk
Gorgias
Front
Notion
Linear
Asana
Trello
Jira

Payments and Documents

Stripe
Square
DocuSign
Dropbox Sign
Google Drive
Dropbox
OneDrive
SharePoint

Future Tool-Calling Capability Map

Current Tools

Calendar availability check
Calendar conflict check
Prepare viewing booking
Confirm viewing booking
Cancel Verbatim-created booking
Send SMS follow-up

Next Tool Capabilities

1 Lead capture and CRM write.

create_lead
update_lead_preferences
add_call_summary_to_crm
create_follow_up_task

1 Listing search.

search_listings
get_listing_details
compare_properties
recommend_properties

1 Knowledge base retrieval.

search_kb
answer_from_kb
cite_internal_source

1 Handoff.

notify_human_agent
send_call_summary_to_slack
create_support_ticket
route_to_team

1 Follow-up.

send_sms
send_whatsapp
send_email
schedule_follow_up

Tool Policy Rules

Every write tool should be classified:

No confirmation needed:
- read-only availability checks
- KB search
- listing search

Soft confirmation:
- send property options
- create low-risk CRM note

Explicit confirmation:

- book viewing
- cancel viewing
- send external message
- create lead with personal details

Never allowed without human:

- delete external records not created by Verbatim
- change payments
- change legal documents
- send sensitive personal data

Future Evaluation Strategy

Evaluation Should Split into Two Systems

The future evaluation system should be split into:

1. Continuous product telemetry
2. Professional non-constant benchmark evaluation

These should not be mixed.

Layer 1 - Continuous Product Telemetry

This runs on every demo or production call.

Purpose:

Debug live behavior
Monitor reliability
Track customer-facing quality
Catch regressions

Metrics:

Transport connected
Mic/audio input seen
STT transcript emitted
STT processing time
LLM provider TTFT
TTS first audio
Playback delay
Perceived latency
Tool call started/completed/failed
Tool duration
Confirmation required/accepted/rejected
Errors
Disconnects
Packet loss/jitter/RTT when browser provides it

This is what the current dashboard is beginning to do.

Layer 2 - Professional Non-Constant Evaluation

This should run:

Before important demos
Before releases
Nightly or weekly
After changing STT/LLM/TTS/tool prompts
After adding new integrations

It should not run on every live call because it can be expensive, slow, and unnatural for production.

Purpose:

Prove system performance
Compare stacks scientifically

Catch conversation/tool regressions
Generate investor/customer credibility

Recommended benchmark families:

Benchmark	Role
VoiceBench	General LLM-based voice assistant evaluation across spoken instruction dimensions
tau2-bench / tau-bench	Tool-agent-user interaction benchmark useful for text/tool policy and task success
EVA-Bench	End-to-end voice-agent evaluation with simulated conversations and voice-specific failure modes
VoiceAgentBench	Agentic spoken task benchmark covering tool use, structure, robustness, and multilingual/adversarial behavior
Internal real-estate benchmark	Domain-specific benchmark for viewings, prices, budget, listings, SMS, CRM, and handoff

Note on "R2-Bench": I did not find a clearly relevant public voice-agent benchmark under that exact name. The closest likely fit for the intended idea is tau2-bench / tau-bench, which focuses on tool-agent-user interaction and task completion. If "R2-Bench" refers to a different private or newer benchmark, it should be added once the exact source is confirmed.

Recommended Evaluation Matrix

A. Core Voice Quality

STT word error rate on domain terms
End-of-turn correctness
Long utterance split rate
Premature assistant start rate
Barge-in handling
Audio cutout count

B. Latency

Clean p50/p95 perceived latency
Real p50/p95 perceived latency
STT processing time
LLM provider TTFT
TTS first audio
Playback delay
Tool-turn latency separate from normal-turn latency

C. Conversation Quality

Answer-first behavior
No repetition of caller's words
No form-like qualification
Correctly suggests property viewing when appropriate
Correctly stops when call is ending
Tone/persona consistency

D. Tool Success

Availability check accuracy
Busy-slot detection
Booking writes exactly once
No double booking
Confirmation policy respected
SMS sent only after confirmation
CRM lead write accuracy
Tool timeout recovery

E. Integration Robustness

- Missing connection fallback
- Expired OAuth fallback
- Provider timeout fallback
- Permission-denied fallback
- Rate-limit fallback
- Secret redaction
- Client isolation

Benchmark Run Types

1 Fast smoke benchmark.

- 10-20 turns
- Run before every live demo
- Goal: no obvious breakage

1 Release benchmark.

- 100-300 simulated calls
- Run before release branches
- Goal: compare stack versions and prompts

1 Professional research benchmark.

- VoiceBench / tau-bench / tau2-bench / EVA-Bench / VoiceAgentBench
- Run weekly or before investor/customer milestones
- Goal: external credibility

1 Domain benchmark.

- Real estate property inquiries
- Viewings
- Price questions
- Budget objections
- Listing search
- CRM capture
- SMS/WhatsApp follow-up
- Human handoff

Evaluation Gating

Before a new integration or prompt can become default:

1. Existing winning stack still works.
2. Normal-turn p95 latency does not regress beyond threshold.
3. Tool-turn success is above threshold.
4. No secret leaks in API/browser logs.
5. No unsafe write without explicit confirmation.
6. Domain benchmark passes.

Recommended Next Steps

Immediate Next Steps

1 Preserve the winning stack.

Do not change:

- LiveKit + Flux + Groq + Sonic-3
- worker isolation
- one active agent
- short prompt
- safe tool surface

1 Make the dashboard clearer for tool calling.

Already started:

terminal shows tool started/succeeded/failed
brief generated call summary
KB input before call

Next:

show latest tool status as a small top-level dashboard badge
show whether a booking is pending, confirmed, failed, or timed out
show whether SMS confirmation was sent

1 Formalize the real-estate demo script.

Create a standard test script:

property price question
specific property interest
booking a viewing
busy slot and alternate suggestion
SMS confirmation
ending the call

1 Add client onboarding schema.

Start with local/JSON/SQLite, not a full SaaS dashboard:

client profile
business type
enabled integrations
connected accounts
allowed tools
tool confirmation rules
KB sources
voice/prompt profile

Next Product Milestone

Build "Client Integration Onboarding v0":

Generate onboarding link
Collect business profile
Select stack
Connect Google Calendar through Nango
Configure SMS sender
Paste/import KB
Generate client tool policy
Start a test call

This is the most important future product move because it converts the current demo into a repeatable onboarding process.

Next Technical Milestone

Build "Evaluation Harness v0":

fixed scenario files
replay runner
mock user simulator
tool backend mocks
scorecards
CSV/JSON outputs
stack comparison reports

The first internal benchmark should cover:

30 calls with no tools
30 calls with calendar booking
30 calls with busy-slot recovery
30 calls with SMS confirmation
30 calls with KB questions

Later Milestones

1 Integration expansion.

- HubSpot lead creation
- Google Sheets/Airtable listing source
- WhatsApp Business follow-up
- Slack/Teams human handoff
- Gmail/Outlook email follow-up

1 KB retrieval.

- dashboard paste remains
- file upload
- Google Drive/Notion sync
- listing inventory sync
- source-aware retrieval

1 Telephony.

- Twilio Voice
- Telnyx
- SIP
- call routing
- multi-call workers
- recording and consent

1 SaaS control plane.

- auth
- client dashboard
- onboarding links
- billing
- worker orchestration
- tenant isolation
- audit logs

Risk Register

Risk	Why it matters	Mitigation
Prompt bloat	Slows Groq and makes agent rigid	Keep global prompt short; use client config and tools for business logic
Raw tool exposure	LLM may misuse third-party APIs	Only expose Verbatim-owned capability tools
Integration sprawl	Too many one-off adapters become unmaintainable	Use Nango first, then adapter registry
Tool latency	Tools can make voice feel slow	Separate normal-turn metrics from tool-turn metrics; use spoken acknowledgement and async fallback
Double booking	Business-critical failure	Always conflict-check before write; idempotency keys; explicit confirmation
Benchmark theater	Nice numbers that do not match real calls	Keep clean p95 and real p95 separate; run domain tests
Telephony premature work	Phone adds noise/latency before WebRTC is proven	Defer until benchmark baseline is solid
Client data leakage	Trust killer	Secret redaction, tenant isolation, no raw OAuth tokens in browser

Updated Roadmap Recommendation

May to early June

Protect winning WebRTC stack
Polish tool dashboard/status
Create onboarding schema
Create evaluation harness v0
Keep adding only high-value tools

June

Client Integration Onboarding v0
Google Calendar + Twilio + KB
HubSpot or Google Sheets/Airtable next
Internal real-estate benchmark

July

Professional evaluation layer
VoiceBench / tau-bench / tau2-bench / EVA-Bench style benchmark runs
Replay system
Stack comparison reports

August

Telephony pilots
Twilio Voice or Telnyx
SIP and routing
Noise and audio real-world handling

September

Multi-client SaaS control plane
Auth
Client dashboard
Onboarding links
Billing
Deployment and worker scaling

Source Notes

Roadmap source:

`/Users/disc/Downloads/_VERBATUM__TECHNICAL_ROADMAP_(MAY__SEPTEMBER).pdf`

Project state sources:

`README.md`
`docs/v2_baseline.md`
`docs/winning_stack_livekit_flux_groq_sonic3.md`
`src/verbatim/`
`static/`
`tests/`

External evaluation and integration references:

- [VoiceBench paper](#)
- [tau-bench paper](#)
- [tau2-bench paper](#)
- [EVA-Bench paper](#)
- [VoiceAgentBench paper](#)
- [Nango documentation](#)

- [Composio documentation](#)
- [Pipedream Connect documentation](#)
- [Zapier AI Actions documentation](#)